

DML Statements

Part 3

Having Clause

- The HAVING clause in SQL filters the data after the GROUP BY clause has grouped it.
- It filters the grouped rows based on a specific condition and is typically used with aggregate functions like SUM, COUNT, AVG, MIN, and MAX.
- The HAVING clause was added to SQL because the WHERE keyword cannot be used with aggregate functions.

HAVING Syntax

- `SELECT column_name(s)`
`FROM table_name`
`WHERE condition`
`GROUP BY column_name(s)`
`HAVING condition`
`ORDER BY column_name(s);`

HAVING Syntax

- `SELECT column_name(s)`
`FROM table_name`
`WHERE condition`
`GROUP BY column_name(s)`
`HAVING condition`
`ORDER BY column_name(s);`

HAVING Syntax

- Example:
- Refer to the table petStore.

13 • **SELECT** * **FROM** petStore;

100%

12:8

Result Grid

Filter Rows:

Search

	pet_id	pet_type	colour	name	cost	
	2	cat	black	Morgan	699	
	3	cat	white	Diamond	1199	
	4	dog	golden	Dolly	1299	
	5	hamster	golden	Sabrina	49	
	6	hamster	brown	Sneak	99	
	7	dog	brown	Flo	799	
	8	fish	golden	Polly	19	
	9	dog	black	Lotte	599	
	10	cat	black	Felix	499	
	11	dog	black		299	

petStore 12

HAVING Syntax

- Example:

```
2 • SELECT pet_type,  
3         COUNT(Name),  
4         SUM(cost),  
5         AVG(cost),  
6         MAX(cost),  
7         MIN(cost)  
8 FROM petStore  
9 GROUP BY pet_type  
10 ORDER BY pet_type;
```

100%24:1

Result Grid

Filter Rows:

Search

Export:

	pet_type	COUNT(Name)	SUM(cost)	AVG(cost)	MAX(cost)	MIN(cost)
	cat	3	2397	799.0000	1199	499
	dog	5	3895	779.0000	1299	299
	fish	1	19	19.0000	19	19
	hamster	2	148	74.0000	99	49

HAVING Syntax

- Example:

```
2 • SELECT pet_type,  
3         COUNT(Name),  
4         SUM(cost),  
5         AVG(cost),  
6         MAX(cost),  
7         MIN(cost)  
8 FROM petStore  
9 GROUP BY pet_type  
10 HAVING AVG(cost)>400  
11 ORDER BY pet_type;
```

10% 24:1

Result Grid   Filter Rows: Export: 

	pet_type	COUNT(Name)	SUM(cost)	AVG(cost)	MAX(cost)	MIN(cost)
▶	cat	3	2397	799.0000	1199	499
	dog	5	3895	779.0000	1299	299

HAVING Syntax

- What is the error?

```
2 • SELECT pet_type,  
3         COUNT(Name),  
4         SUM(cost),  
5         AVG(cost),  
6         MAX(cost),  
7         MIN(cost)  
8 FROM petStore  
9 GROUP BY pet_type  
10 HAVING cost>400  
11 ORDER BY pet_type;
```

Action					Response
SELECT pet_type, COUNT(Name), SUM(cost), AVG(cost), M...					Error Code: 1054. Unknown column 'cost' in 'having clause'
SELECT pet_type, COUNT(Name), SUM(cost), AVG(cost), M...					4 row(s) returned
SELECT pet_type, COUNT(Name), SUM(cost), AVG(cost), M...					2 row(s) returned
SELECT pet_type, COUNT(Name), SUM(cost), AVG(cost), M...					2 row(s) returned
SELECT pet_type, COUNT(Name), SUM(cost), AVG(cost), M...					Error Code: 1054. Unknown column 'cost' in 'having clause'

Having clause can only be used with aggregate functions.

HAVING Syntax

- To address the error:

```
2 • SELECT pet_type,  
3         COUNT(Name),  
4         SUM(cost),  
5         AVG(cost),  
6         MAX(cost),  
7         MIN(cost)  
8 FROM petStore  
9 GROUP BY pet_type  
10 HAVING AVG(cost)>400  
11 ORDER BY pet_type;
```

100% 19:11

Result Grid   Filter Rows: Export: 

pet_type	COUNT(Name)	SUM(cost)	AVG(cost)	MAX(cost)	MIN(cost)
cat	3	2397	799.0000	1199	499
dog	5	3895	779.0000	1299	299

Having clause can only be used with aggregate functions.

HAVING Syntax

- Example 2

```
2 • SELECT pet_type,  
3       COUNT(Name),  
4       SUM(cost),  
5       AVG(cost),  
6       MAX(cost),  
7       MIN(cost)  
8 FROM petStore  
9 GROUP BY pet_type  
10 HAVING AVG(cost)>400  
11 ORDER BY pet_type;
```

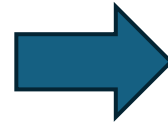
100%  17:1

Result Grid  

Filter Rows:

Export: 

	pet_type	COUNT(Name)	SUM(cost)	AVG(cost)	MAX(cost)	MIN(cost)
▶	cat	3	2397	799.0000	1199	499
	dog	5	3895	779.0000	1299	299



```
2 • SELECT pet_type,  
3       COUNT(Name),  
4       SUM(cost),  
5       AVG(cost),  
6       MAX(cost),  
7       MIN(cost)  
8 FROM petStore  
9 GROUP BY pet_type  
10 HAVING AVG(cost)>400 AND SUM(cost)<2500  
11 ORDER BY pet_type;
```

100% 20:1

Result Grid Filter Rows: Search Export:

	pet_type	COUNT(Name)	SUM(cost)	AVG(cost)	MAX(cost)	MIN(cost)
▶	cat	3	2397	799.0000	1199	499

You can incorporate multiple conditions using the AND and OR keyword.

Note to Remember:

- To avoid having errors from using the different clauses, aside from following the syntax, ensure that the data type is appropriate.
- You can check the data type in MySQL Workbench, from this section:

The screenshot displays the MySQL Workbench interface. On the left, the 'DBPets' database is expanded, showing the 'petStore' table with columns: pet_id, pet_type, colour, name, and cost. The 'cost' column is selected. Below the schema tree, the 'Object Info' tab is active, showing the definition of the 'cost' column as 'int'. On the right, a SQL query is executed, and the 'Result Grid' shows the following data:

pet_type	COUNT(Name)	SUM(cost)
cat	3	2397
dog	5	3895

The SQL query used is:

```
2 • SELECT pet_type,  
3         COUNT(Name),  
4         SUM(cost),  
5         AVG(cost),  
6         MAX(cost),  
7         MIN(cost)  
8 FROM petStore  
9 GROUP BY pet_type  
10 HAVING AVG(cost)>400  
11 ORDER BY pet_type;
```

Inserting Data

MySQL INSERT INTO Statement

- The INSERT INTO statement is used to insert new records in a table. It is possible to write the INSERT INTO statement in two ways:

1. Specify both the column names and the values to be inserted:

2. If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is in the same order as the columns in the table.

MySQL INSERT INTO Statement

Syntax:

- **INSERT INTO** *table_name*
VALUES (*value1*, *value2*, *value3*, ...);

MySQL INSERT INTO Statement

- The following SQL statement inserts a new record in the "tbl_studentInfo" table:

```
• INSERT INTO tbl_studentInfo (  
    studentID,  
    studentName,  
    Gender,  
    Course,  
    yearLevel)  
  
  VALUES (  
    '1001',  
    'Juan dela Cruz',  
    'Male',  
    'Course',  
    '1');
```

MySQL INSERT INTO Statement

```
= INSERT INTO tblemployee(  
  employeeID,  
  empName,  
  empPosition,  
  Department,  
  empSalary,  
  performanceRating)
```

```
= VALUES (  
  '2',  
  'Pedro Penduko',  
  'Programmer',  
  'IT',  
  80000,  
  'Very Satisfactory');
```

```
INSERT INTO dbhr.tblemployee
```

```
= VALUES (  
  '3',  
  'Maria Reyes',  
  'Full Stack Developer',  
  'IT',  
  80000,  
  'Excellent');
```


MySQL INSERT INTO Statement

```
INSERT INTO dbhr.tblemployee
```

```
= VALUES (  
  '3',  
  'Maria Reyes',  
  'Full Stack Developer',  
  'IT',  
  80000,  
  'Excellent');
```

```
= INSERT INTO tblemployee(  
  employeeID,  
  empName,  
  Department,  
  empSalary,  
  performanceRating)
```

```
= VALUES (  
  '4',  
  'Shirley Ramirez',  
  'IT',  
  65000,  
  'Excellent');
```

MySQL INSERT INTO Statement

```
INSERT INTO dbhr.tblemmployee
```

```
VALUES
```

```
('5','Jose Chua','Software Engineer','IT',75000,'Very Satisfactory'),  
( '6','Christina Lee','Front End Developer','IT',75000,'Very Satisfactory'),  
( '7','Josh Christobal','Programmer','IT',83000,'Satisfactory');
```

Updating Data

The SQL UPDATE Statement

- The UPDATE statement is used to modify the existing records in a table.

Update Syntax

- `UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;`

Few reminders before using the UPDATE

- If you encounter this kind of error after running the update statement:

Sample Code:

```
UPDATE petStore SET name="Sammy" WHERE pet_id=11
```

- *“Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE that uses a KEY column. To disable safe mode, toggle the option in Preferences -> SQL Editor and reconnect.”*

Few reminders before using the UPDATE

- MySQL Error Code 1175 happens when you try to update or delete rows in a table without a specified WHERE clause or with a WHERE clause that doesn't use a key column.
- This safety feature prevents accidental modifications of multiple rows in production databases.

Few reminders before using the UPDATE

- **Understanding Error Code 1175**
- Error Code 1175 is triggered by the SQL_SAFE_UPDATES mode, which is enabled by default in MySQL.
- It requires that any UPDATE or DELETE operation must include a key column in the WHERE clause or be limited using the LIMIT clause.

Few reminders before using the UPDATE

Resolving Error Code 1175

- **DISABLING SQL_SAFE_UPDATES TEMPORARILY**
- To perform the operation, you can temporarily disable SQL_SAFE_UPDATES:
- `SET SQL_SAFE_UPDATES = 0;`
- `-- Perform your UPDATE or DELETE operations here`
- `SET SQL_SAFE_UPDATES = 1;`
- **Set the value to 1 again to enable SQL_SAFE_UPDATES**

Disabling and Enabling SQL_SAFE_UPDATES

```
13 • SET SQL_SAFE_UPDATES = 0;
14   -- Perform your UPDATE or DELETE operations here
15
16 • UPDATE petStore
17   SET name="Sammy"
18   WHERE pet_id=11;
19
20 • SELECT * FROM petStore;
21 • SET SQL_SAFE_UPDATES = 1;
```

Example

13 • `SELECT * FROM petStore;`

100%

12:8

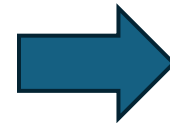
Result Grid

Filter Rows:

Search

	pet_id	pet_type	colour	name	cost	
	2	cat	black	Morgan	699	
	3	cat	white	Diamond	1199	
	4	dog	golden	Dolly	1299	
	5	hamster	golden	Sabrina	49	
	6	hamster	brown	Sneak	99	
	7	dog	brown	Flo	799	
	8	fish	golden	Polly	19	
	9	dog	black	Lotte	599	
	10	cat	black	Felix	499	
	11	dog	black		299	

petStore 12



16 • `UPDATE petStore`
17 `SET name="Sammy"`
18 `WHERE pet_id=11;`
19

100%  26:13

Result Grid   Filter Rows:

	pet_id	pet_type	colour	name	cost
▶	1	dog	black	Buster	899
	2	cat	black	Morgan	699
	3	cat	white	Diamond	1199
	4	dog	golden	Dolly	1299
	5	hamster	golden	Sabrina	49
	6	hamster	brown	Sneak	99
	7	dog	brown	Flo	799
	8	fish	golden	Polly	19
	9	dog	black	Lotte	599
	10	cat	black	Felix	499
	11	dog	black	Sam	299

UPDATE Multiple Records

- It is the WHERE clause that determines how many records will be updated.

```
28  -- Update all costs of dogs to 899.  
29  • UPDATE petStore  
30  SET cost=899  
31  WHERE pet_type="dog";
```

100%	⬆	22:31		
Action Output ⬆				
		Time	Action	Response
✓	40	09:28:52	SELECT * FROM petStore LIMIT 0, 50	11 row(s) returned
✓	41	09:36:10	UPDATE petStore SET cost=899 WHERE pet...	4 row(s) affected Rows matched: 5 Changed: 4 Warnings: 0

UPDATE Multiple Records

- Update WARNING
- Be careful when updating records.
- If you omit the WHERE clause, ALL records will be updated!

UPDATE Multiple Records

13 • `SELECT * FROM petStore;`

100%  12:8

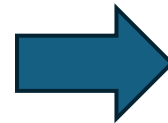
Result Grid 

 Filter Rows:

 Search

	pet_id	pet_type	colour	name	cost	
	2	cat	black	Morgan	699	
	3	cat	white	Diamond	1199	
	4	dog	golden	Dolly	1299	
	5	hamster	golden	Sabrina	49	
	6	hamster	brown	Sneak	99	
	7	dog	brown	Flo	799	
	8	fish	golden	Polly	19	
	9	dog	black	Lotte	599	
	10	cat	black	Felix	499	
	11	dog	black		299	

petStore 12



28 `-- Update all costs of dogs to 899.`
29 • `UPDATE petStore`
30 `SET cost=899`
31 `WHERE pet_type="dog";`
32
33 • `SELECT * FROM petStore;`

100%

12:26

Result Grid

Filter Rows:

Search

E

	pet_id	pet_type	colour	name	cost	
▶	1	dog	black	Buster	899	
	2	cat	black	Morgan	699	
	3	cat	white	Diamond	1199	
	4	dog	golden	Dolly	899	
	5	hamster	golden	Sabrina	49	
	6	hamster	brown	Sneak	99	
	7	dog	brown	Flo	899	
	8	fish	golden	Polly	19	
	9	dog	black	Lotte	899	
	10	cat	black	Felix	499	
	11	dog	black	Sam	899	

Deleting Data

The MySQL DELETE Statement

- The DELETE statement is used to delete existing records in a table.

- DELETE Syntax

DELETE FROM *table_name* **WHERE** *condition*;

The MySQL DELETE Statement

Important points to remember:

- Be careful when deleting records in a table! Notice the WHERE clause in the DELETE statement.
- The WHERE clause specifies which record(s) should be deleted.
- ***If you omit the WHERE clause, all records in the table will be deleted!***

The MySQL DELETE Statement

```
33 • SELECT * FROM petStore;
```

100%

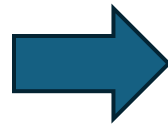
1:27

Result Grid

Filter Rows:

Search

	pet_id	pet_type	colour	name	cost
	1	dog	black	Buster	899
	2	cat	black	Morgan	699
	3	cat	white	Diamond	1199
	4	dog	golden	Dolly	899
	5	hamster	golden	Sabrina	49
	6	hamster	brown	Sneak	99
	7	dog	brown	Flo	899
	8	fish	golden	Polly	19
▶	9	dog	black	Lotte	899
	10	cat	black	Felix	499
	11	dog	black	Sam	899



```
33 • SELECT * FROM petStore;  
34 • DELETE FROM petStore WHERE name='Sam';  
35
```

100%	↕	29:23	
Action Output			
	Time	Action	Response
✓ 43	09:46:30	DELETE FROM petStore WHERE name='Sam'	1 row(s) affected
✓ 44	09:46:43	DELETE FROM petStore WHERE name='Sam'	0 row(s) affected

```
36 • SELECT * FROM petStore;
```

```
37
```

100%  15:29

Result Grid   Filter Rows:

	pet_id	pet_type	colour	name	cost
▶	1	dog	black	Buster	899
	2	cat	black	Morgan	699
	3	cat	white	Diamond	1199
	4	dog	golden	Dolly	899
	5	hamster	golden	Sabrina	49
	6	hamster	brown	Sneak	99
	7	dog	brown	Flo	899
	8	fish	golden	Polly	19

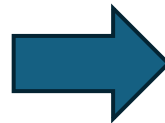
The MySQL DELETE Statement

```
33 • SELECT * FROM petStore;  
34 • DELETE FROM petStore WHERE name='Sam';  
35
```

100% 29:23

Action Output

	Time	Action	Response
✓ 43	09:46:30	DELETE FROM petStore WHERE name='Sam'	1 row(s) affected
✓ 44	09:46:43	DELETE FROM petStore WHERE name='Sam'	0 row(s) affected



```
36 • SELECT * FROM petStore;  
37
```

100%

15:29

Result Grid

Filter Rows:

Search

	pet_id	pet_type	colour	name	cost
	1	dog	black	Buster	899
	2	cat	black	Morgan	699
	3	cat	white	Diamond	1199
	4	dog	golden	Dolly	899
	5	hamster	golden	Sabrina	49
	6	hamster	brown	Sneak	99
	7	dog	brown	Flo	899
	8	fish	golden	Polly	19
	9	dog	black	Lotte	899
	10	cat	black	Felix	499

Delete All Records

- Deleting all rows in a table is possible without deleting the table. This means that the table structure, attributes, and indexes will be intact:
- `DELETE FROM table_name;`

Delete Records in a Column

- The following SQL statement deletes all rows in the "Customers" table, without deleting the table:

Example:

- `DELETE FROM Customers;`

Activity:

- From your petStore table, add the following records:

Pet ID= 11

Pet Type: Fish

Colour: Black

Name: Felix

Cost: 89

Activity:
Write the
equivalent code.

From your petStore table, add the following records:

Pet ID= 11
Pet Type: Fish
Colour: Black
Name: Felix
Cost: 89

From your pet store table, add the following records:

Pet ID= 12
Pet Type: Hamster
Colour: Brown
Name: Hammy
Cost: 199

From your petStore table, modify the following records:

Pet ID= 11
Pet Type: Fish
Colour: Black
Name: Feliz
Cost: 89

From your petStore table, delete all records of pet type fish.
From your petStore table, update the costs of all cats to 699.